

Chest X-Ray Pneumonia Classifier (Kaggle Optimized)

This notebook trains a transfer-learning classification model on the `paultimothymooney/chest-xray-pneumonia` dataset.

Important: Ensure you have attached the dataset to this notebook by clicking **Add Data** on the right panel and searching for `chest-xray-pneumonia`.

1. Install & Import Dependencies

Kaggle pre-installs most libraries like PyTorch, OpenCV, and Matplotlib. We only need to install a few specific tracking and visualization tools.

```
!pip install -q mlflow grad-cam torchinfo

import os
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import cv2

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, models, transforms
from torchinfo import summary

import mlflow
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, roc_curve, auc

from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
from pytorch_grad_cam.utils.image import show_cam_on_image

# Set GPU awareness (Kaggle provides P100/T4 GPUs)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
```

2. Dataset Paths and Exploration

We point our directories to Kaggle's native `/kaggle/input/` directory, visualize the class balance, and look at sample images.

```
# Kaggle native dataset path
data_dir = '/kaggle/input/chest-xray-pneumonia/chest_xray'

train_dir = os.path.join(data_dir, 'train')
val_dir = os.path.join(data_dir, 'val')
test_dir = os.path.join(data_dir, 'test')

classes = ['NORMAL', 'PNEUMONIA']

def get_class_distributions(directory):
    counts = {}
    for c in classes:
        path = os.path.join(directory, c)
        if os.path.exists(path):
            counts[c] = len(os.listdir(path))
    return counts

train_counts = get_class_distributions(train_dir)
test_counts = get_class_distributions(test_dir)

# Plot Class Distributions
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.barplot(x=list(train_counts.keys()),
            y=list(train_counts.values()))
plt.title('Training Class Distribution')

plt.subplot(1, 2, 2)
sns.barplot(x=list(test_counts.keys()), y=list(test_counts.values()))
plt.title('Testing Class Distribution')
plt.show()

# Visualize Sample Images
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
for i, cls in enumerate(classes):
    img_path = os.path.join(train_dir, cls,
                             os.listdir(os.path.join(train_dir, cls))[0])
    img = Image.open(img_path)
    axes[i].imshow(img, cmap='gray')
    axes[i].set_title(f'Sample: {cls}')
    axes[i].axis('off')
plt.show()
```

3. Data Loading and Transformations

Applying resizing, augmentations to prevent overfitting, and ImageNet normalization.

```
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

image_datasets = {
    'train': datasets.ImageFolder(train_dir, train_transforms),
    'val': datasets.ImageFolder(val_dir, val_test_transforms),
    'test': datasets.ImageFolder(test_dir, val_test_transforms)
}

dataloaders = {
    'train': DataLoader(image_datasets['train'], batch_size=32,
        shuffle=True, num_workers=2),
    'val': DataLoader(image_datasets['val'], batch_size=32,
        shuffle=False, num_workers=2),
    'test': DataLoader(image_datasets['test'], batch_size=32,
        shuffle=False, num_workers=2)
}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val', 'test']}
class_names = image_datasets['train'].classes

print(f'Detected Classes: {class_names}')
print(f'Dataset Split Sizes: {dataset_sizes}')
```

4. Transfer Learning Model Build

Using DenseNet121 with a custom classifier head.

```
def build_model():
    model =
models.densenet121(weights=models.DenseNet121_Weights.DEFAULT)
```



```

labels.to(device)
        optimizer.zero_grad()

        with torch.set_grad_enabled(phase == 'train'):
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            loss = criterion(outputs, labels)

            if phase == 'train':
                loss.backward()
                optimizer.step()

            running_loss += loss.item() * inputs.size(0)
            running_corrects += torch.sum(preds ==
labels.data)

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double() /
dataset_sizes[phase]

            print(f'{phase.capitalize()} Loss: {epoch_loss:.4f}
Acc: {epoch_acc:.4f}')
            mlflow.log_metric(f'{phase}_loss', epoch_loss,
step=epoch)
            mlflow.log_metric(f'{phase}_acc', epoch_acc.item(),
step=epoch)

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())
            print()

            print(f'Best Validation Accuracy: {best_acc:.4f}')
            model.load_state_dict(best_model_wts)

            # Save in Kaggle working directory
            torch.save(model.state_dict(),
'/kaggle/working/best_model.pth')

        return model

# Set num_epochs=3 for a quick run. Increase for better results.
model = train_model(model, criterion, optimizer, num_epochs=3)

```

6. Model Evaluation

Evaluate on the unseen test set and plot metrics.

```

model.eval()
all_preds = []
all_labels = []
all_probs = []

with torch.no_grad():
    for inputs, labels in dataloaders['test']:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs[:, 1].cpu().numpy())

acc = accuracy_score(all_labels, all_preds)
prec = precision_score(all_labels, all_preds)
rec = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)

print(f'Test Accuracy: {acc:.4f}')
print(f'Test Precision: {prec:.4f}')
print(f'Test Recall: {rec:.4f}')
print(f'Test F1-Score: {f1:.4f}')

cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()

fpr, tpr, _ = roc_curve(all_labels, all_probs)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC)')
plt.legend(loc='lower right')
plt.show()

```

7. Grad-CAM Interpretability

```
target_layers = [model.features[-1]]
cam = GradCAM(model=model, target_layers=target_layers)

def visualize_gradcam(image_path, label_idx):
    img_pil = Image.open(image_path).convert('RGB')
    input_tensor =
val_test_transforms(img_pil).unsqueeze(0).to(device)

    model.eval()
    output = model(input_tensor)
    pred_idx = output.argmax(dim=1).item()

    targets = [ClassifierOutputTarget(label_idx)]
    grayscale_cam = cam(input_tensor=input_tensor, targets=targets)[0,
:]

    img_cv = cv2.imread(image_path)
    img_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)
    img_cv = cv2.resize(img_cv, (224, 224))
    img_float = np.float32(img_cv) / 255

    cam_image = show_cam_on_image(img_float, grayscale_cam,
use_rgb=True)

    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(img_cv)
    plt.title('Original X-ray Image')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(cam_image)
    plt.title(f'Grad-CAM (Pred: {class_names[pred_idx]})')
    plt.axis('off')
    plt.show()

sample_img_path = os.path.join(test_dir, 'PNEUMONIA',
os.listdir(os.path.join(test_dir, 'PNEUMONIA'))[0])
visualize_gradcam(sample_img_path, label_idx=1)
```

8. Single Image Inference Function

```
def predict_single_image(image_path, saved_model_path, device):
    infer_model = build_model()
    infer_model.load_state_dict(torch.load(saved_model_path,
map_location=device))
    infer_model.eval()
    infer_model.to(device)
```

```

img_pil = Image.open(image_path).convert('RGB')
input_tensor =
val_test_transforms(img_pil).unsqueeze(0).to(device)

with torch.no_grad():
    outputs = infer_model(input_tensor)
    probs = torch.softmax(outputs, dim=1)
    conf, pred = torch.max(probs, 1)

pred_class = class_names[pred.item()]
confidence = conf.item()

print(f'Prediction: {pred_class} (Confidence: {confidence:.2%})')
return pred_class, confidence

pred, conf = predict_single_image(sample_img_path,
'/kaggle/working/best_model.pth', device)

```

9. Save as TorchScript for FastAPI

```

model.eval()
dummy_input = torch.randn(1, 3, 224, 224).to(device)
traced_model = torch.jit.trace(model, dummy_input)

export_path = '/kaggle/working/densenet121_pneumonia_prod.pt'
traced_model.save(export_path)

print(f'TorchScript model saved to: {export_path}')
print('Download this file from the Kaggle output panel to deploy
locally with FastAPI.')

```